

python Strings

- Strings in python are surrounded by either single quotation marks, or double quotation marks.
- 'hello' is the same as "hello".

Example:

- My_string="hello"
- My_string='hello'

Examp2:

```
print("It's alright")
```

```
print("He is called 'John' ")
```

```
print('He is called "John"')
```

How to assign string to a variable

- A="hi"

1.Multiline strings

- a multiline string to a variable by using three quotes:

Example:

```
A=""" CNN are the computer vision approach utilized in Chandrayaan-3 to analyze photos and films from the lander's camera. CNNs are a sort of deep learning system that excels at image recognition. CNNs can recognize picture elements such as edges, forms, and textures. This data can be utilized to decipher the meaning of an image or video."""
```

2.Python - Slicing Strings:

you can return a range of characters by using the slice syntax.

Syntax: str_object[start_pos:pos:step]

Example:

```
s = 'HelloWorld'
```

```
print(s[:])
```

```
print(s[::])
```

Example 2:

```
S="helloworld"
```

```
First_five_char=s[:5]
```

Example3:

```
third_to_fifth_chars = s[2:5]
```

```
print(third_to_fifth_chars)
```

Reverse a String using Slicing

- We can reverse a string using slicing by providing the step value as -1.

Example:

```
original_string="Good"
```

```
reversed_string=original_string[::-1]
```

```
Print(reversed_string)
```

Python String methods

Python strings are a sequence of characters used to store and manipulate text. They are immutable, meaning once created, their content cannot be changed; it can only be replaced. Python offers a wide range of built-in methods for string handling, making it easy to perform operations like formatting, searching, and slicing. These features make strings a crucial part of Python programming for tasks ranging from simple message displays to complex data processing.

Case Changing of Strings

Python provides several built-in methods for changing the case of strings, which are particularly useful in data processing and user interface development. These methods do not change the original string but return a new string with the applied changes, adhering to Python's string immutability.

1. **.lower():** Converts all characters in the string to lowercase. This method is widely used in case-insensitive comparisons.
2. **.upper():** Transforms all characters to uppercase. It's commonly used for emphasizing certain texts or for uniform data entry.
3. **.capitalize():** Capitalizes the first character of the string while turning the rest into lowercase. Useful for formatting titles or names.
4. **.title():** Converts the first character of every word to uppercase and the other characters to lowercase. Ideal for formatting headings or titles.
5. **.swapcase():** Reverses the case of all characters in the string, turning lowercase letters to uppercase and vice versa.
6. **.casefold():** Similar to .lower(), but more aggressive. It's designed to remove all case distinctions in a string and is often used for more effective case-insensitive comparisons.

These methods make string handling in Python not only effective but also intuitive, catering to a wide array of case manipulation needs in programming. Let's take an example to understand the above functions.

Example

Consider the string 'PyThOn Is AwEsOmE'. We'll apply various case-changing methods to this string and observe the outputs:

```
original_string = 'PyThOn Is AwEsOmE'

# Applying .lower()
lowercase = original_string.lower()
print('lower:', lowercase)

# Applying .upper()
uppercase = original_string.upper()
print('upper:', uppercase)

# Applying .capitalize()
capitalized = original_string.capitalize()
print('capitalize:', capitalized)

# Applying .title()
titlecase = original_string.title()
print('title:', titlecase)

# Applying .swapcase()
swapcased = original_string.swapcase()
print('swapcase:', swapcased)

# Applying .casefold()
casefolded = original_string.casefold()
print('casefold:', casefolded)
```

Output:

```
lower: python is awesome
upper: PYTHON IS AWESOME
capitalize: Python is awesome
title: Python Is Awesome
swapcase: pYtHoN iS aWeSoMe
casefold: python is awesome
```

In this example, each method is applied to the same string, showing how it affects the case of the characters. These methods are essential for various text processing tasks, from formatting user input to ensuring case-insensitive comparisons in data analysis.

Python String Methods

Here's a table format summary of Python string methods with a one-liner description for each:

Method	Description
<code>capitalize()</code>	Capitalizes the first character and lowercase the rest.
<code>casefold()</code>	Converts the string to lowercase for caseless matching.
<code>center()</code>	Centers the string within a specified width.
<code>count()</code>	Counts occurrences of a substring in the string.
<code>encode()</code>	Converts the string to a specified encoding format.
<code>endswith()</code>	Verifies whether the string concludes with the given suffix.
<code>expandtabs()</code>	Replaces tab characters with spaces.
<code>find()</code>	Provides the smallest index where the specified substring is located, if present.
<code>format()</code>	Formats the string into a nicer output.
<code>format_map()</code>	Similar to <code>format()</code> , but accepts a dictionary.
<code>index()</code>	Returns the lowest index of the substring and raises an error if not found.
<code>isalnum()</code>	Checks if all characters are alphanumeric.
<code>isalpha()</code>	Checks if all characters are alphabetic.
<code>isdecimal()</code>	Checks if all characters are decimals.
<code>isdigit()</code>	Checks if all characters are digits.
<code>isidentifier()</code>	Checks if the string is a valid identifier.
<code>islower()</code>	Checks if all characters are lowercase.
<code>isnumeric()</code>	Checks if all characters are numeric.
<code>isprintable()</code>	Checks if all characters are printable.
<code>isspace()</code>	Checks if all characters are whitespace.
<code>istitle()</code>	Checks if the string is titled (capitalized words).
<code>isupper()</code>	Checks if all characters are uppercase.
<code>join()</code>	Joins iterable elements with the string as a separator.
<code>ljust()</code>	Left-justifies in a specified width field.
<code>lower()</code>	Converts all characters to lowercase.
<code>lstrip()</code>	Trims leading whitespace.

Method	Description
<code>maketrans()</code>	Creates a translation table for <code>translate()</code> method.
<code>partition()</code>	Divides the string at the initial instance of the separator.
<code>replace()</code>	Substitutes instances of one substring with a different one.
<code>rfind()</code>	Returns the highest index of the substring.
<code>rindex()</code>	Returns the highest index of the substring, raises an error if not found.
<code>rjust()</code>	Right-justifies in a specified width field.
<code>rpartition()</code>	Splits the string at the last occurrence of the separator.
<code>rsplit()</code>	Splits the string at the separator from the right.
<code>rstrip()</code>	Trims trailing whitespace.
<code>split()</code>	Splits the string at the separator.
<code>splitlines()</code>	Splits the string at line breaks.
<code>startswith()</code>	Checks if the string starts with the specified prefix.
<code>strip()</code>	Trims leading and trailing whitespace.
<code>swapcase()</code>	Swaps the case of all characters.
<code>title()</code>	Converts the first character of each word to uppercase.
<code>translate()</code>	Transforms the string using a translation table.
<code>upper()</code>	Converts all characters to uppercase.
<code>zfill()</code>	Pads the string with zeros on the left to fill a width.

This table serves as a quick reference guide for Python string methods, providing a brief description of what each method does.

Examples

Using String Methods in Python

Python provides many built-in string methods that help with manipulation and processing. Here are some commonly used ones with examples:

1. Case Conversion

```
s = "hello World"
```

```
print(s.upper()) # HELLO WORLD
```

```
print(s.lower()) # hello world
```

```
print(s.title()) # Hello World
```

```
print(s.capitalize()) # Hello world
```

```
print(s.swapcase()) # HELLO wORLD
```

2. Checking String Properties

```
s = "Python123"
```

```
print(s.isalpha()) # False (contains numbers)
```

```
print(s.isdigit()) # False (not all characters are digits)
```

```
print("12345".isdigit()) # True
```

```
print(s.isalnum()) # True (contains letters and numbers)
```

```
print(" ".isspace()) # True (only spaces)
```

```
print("Python".islower()) # False
```

```
print("python".islower()) # True
```

```
print("HELLO".isupper()) # True
```

3. Searching & Finding

```
s = "Python is awesome"
```

```
print(s.find("is"))    # 7 (Returns index of first occurrence)
```

```
print(s.index("is"))   # 7 (Same as find, but raises an error if not found)
```

```
print(s.rfind("o"))    # 10 (Finds last occurrence)
```

```
print(s.count("o"))    # 2 (Counts occurrences of "o")
```

```
print(s.startswith("Py")) # True
```

```
print(s.endswith("me")) # True
```

4. Modifying Strings

```
s = " Hello Python "
```

```
print(s.strip()) # "Hello Python" (Removes spaces from both ends)
```

```
print(s.lstrip()) # "Hello Python " (Removes spaces from left)
```

```
print(s.rstrip()) # " Hello Python" (Removes spaces from right)
```

```
s2 = "I love Python"
```

```
print(s2.replace("love", "like")) # "I like Python"
```

5. Splitting & Joining

```
s = "apple,banana,orange"
```

```
# Split a string into a list
```

```
words = s.split(",")  
  
print(words) # ['apple', 'banana', 'orange']
```

```
# Join a list into a string  
  
new_s = "-".join(words)  
  
print(new_s) # "apple-banana-orange"
```

6. Formatting Strings

```
name = "Alice"  
  
age = 25  
  
print(f"My name is {name} and I am {age} years old.")  
  
# Output: My name is Alice and I am 25 years old.
```

```
print("I have {} apples and {} bananas".format(5, 3))  
  
# Output: I have 5 apples and 3 bananas
```

```
print("The price is {:.2f}".format(45.6789))  
  
# Output: The price is 45.68 (2 decimal places)
```

7. Reversing a String

```
s = "Python"  
  
print(s[::-1]) # nohtyP
```

Conclusion

- Python strings, as sequences of characters, are versatile and essential in various applications, from data manipulation to user interface design.
- The immutability of Python strings ensures that original data is not altered unintentionally, with string methods returning new string objects.
- Understanding and effectively utilizing these string methods can significantly enhance coding efficiency and data processing capabilities in Python.
- The simplicity and readability of Python's string methods make them accessible to programmers at all levels, from beginners to experts.
- Regular practice and application of these string methods in real-world scenarios will deepen a developer's proficiency in Python and its applications in text processing.

1.len()

- `s="hello"`
- `Print(len(s))`

2.count()

`Print(s.count("a"))`

3.lower()

`s.lower()`

4.upper()

5.find()

Ex: `s.find("0")`

6.strip() : This method removes leading and trailing whitespaces from a string.

`S=" hello "`

`Print(s.strip())`

7.replace()

Ex: `print(s.replace("h","j"))`

8.split()

`Print(s.split(","))`

9.Concatenate

`string1+string2`

10.capitalize()

String slicing

in Python is a way to get specific parts of a string by using start, end, and step values. It's especially useful for text manipulation and data parsing.

Let's take a quick example of string slicing:

```
s = "Hello, Python!"
```

```
# Slice string from index 0 to index 5 (exclusive)
```

```
s2 = s[0:5]
```

```
print(s2)
```

Syntax of String Slicing in Python

substring = s[start : end : step]

Parameters:

- **s**: The original string.
- **start (optional)**: Starting index (inclusive). Defaults to **0** if omitted.
- **end (optional)**: Stopping index (exclusive). Defaults to the end of the string if omitted.
- **step (optional)**: Interval between indices. A positive value slices from left to right, while a negative value slices from right to left. If omitted, it defaults to 1 (no skipping of characters).

Retrieve All Characters

To retrieve the entire string, use slicing without specifying any parameters.

```
s = "Hello, World!"
```

```
# Get the entire string
```

```
s2 = s[:]
```

```
s3 = s[::-1]
```

```
print(s2)
```

```
print(s3)
```

Explanation: Using `[:]` or `[::-1]` without specifying **start**, **end**, or **step** returns the complete string.

Get All Characters Before or After a Specific Position

To get all the items from a specific position to the end of the string, we can specify the **start** index and leave the **end** blank.

And to get all the items before a specific index, we can specify the **end** index while leaving start blank.

```
s = "Hello, World!"

# Characters from index 7 to the end
print(s[7:])

# Characters from the start up to index 5 (exclusive)
print(s[:5])
```

Explanation: The slice `s[7:]` starts from index 7 and continues to the end of the string.

Extract Characters Between Two Positions

To extract characters between specific positions, provide both **start** and **end** indices.

```
s = "Hello, World!"

# Characters from index 1 to index 5 (excluding 5)
print(s[1:5])
```

Get Characters at Specific Intervals

To retrieve characters at regular intervals, use the **step** parameter.

```
s = "abcdefghi"

# Every second character
print(s[::2])

# Every third character from index 1 to 8 (exclusive)
print(s[1:8:3])
```

Explanation: The slice `s[::2]` takes every second character from the string.

Using Negative Indexing in Slicing

Negative indexing is useful for accessing elements from the end of the String. The last element has an index of -1, the second last element -2, and so on.

Extract Characters Using Negative Indices

Below example shows how to use negative numbers to access elements from the string starting from the end. Negative indexing makes it easy to get items without needing to know the exact length of the string.

```
s = "abcdefghijklmno"

# Characters from index -4 to the end
print(s[-4:])

# Characters from the start up to index -3 (excluding -3)
print(s[:-3])

# Characters from index -5 to -2 (excluding -2)
print(s[-5:-2])

# Get every 2nd elements from index -8 to -1 (excluding index -1)
print(s[-8:-1:2])
```

Reverse a String Using Slicing

To reverse a string, use a **negative step value of -1**, which moves from the end of the string to the beginning.

```
s = "Python"

# Reverse the string
print(s[::-1])
```

Explanation: The slice **s[::-1]** starts from the end and steps backward through the string, which effectively reversing it. This method does not alter the original string.

Examples

Python String Slicing Examples

String slicing allows you to extract specific portions of a string using the syntax:

string[start:stop:step]

Let's explore some examples:

1. Basic Slicing

```
s = "PythonProgramming"
```

```
print(s[0:6]) # Output: Python
```

```
print(s[6:18]) # Output: Programming
```

2. Omitting Start or Stop

```
print(s[:6]) # Output: Python (from start to index 5)
```

```
print(s[6:]) # Output: Programming (from index 6 to end)
```

```
print(s[:]) # Output: PythonProgramming (entire string)
```

3. Using Step (Skipping Characters)

```
print(s[0:12:2]) # Output: PtoPorm (every 2nd character)
```

```
print(s[::-1]) # Output: gnimmargorPnohtyP (reversed string)
```

```
print(s[::-2]) # Output: gimroPhy (every 2nd character in reverse)
```

4. Negative Indexing

```
print(s[-10:-1]) # Output: ogrammin (excludes last character)
```

```
print(s[-6:]) # Output: mming (last 6 characters)
```

5. Extracting Specific Parts

```
date = "2025-03-17"
```

```
print(date[:4]) # Output: 2025 (Year)
```

```
print(date[5:7]) # Output: 03 (Month)
```

```
print(date[8:]) # Output: 17 (Day)
```

